Picker, Toggle et contrôles de sélection en SwiftUI

420-249-AH - Programmation mobile iOS

1. Pourquoi utiliser des contrôles de sélection ?

Dans une application mobile, plusieurs valeurs ne devraient pas être saisies manuellement : une catégorie, une priorité, un statut, un mode d'affichage, l'activation des notifications. Des contrôles dédiés donnent des valeurs cohérentes et plus faciles à valider.

Contrôle	Utilisation
`Picker`	Choisir une valeur parmi plusieurs
`Toggle`	Une valeur vrai ou faux
`Stepper`	Une valeur numérique ajustable
`DatePicker`	Une date ou une heure

2. Picker

Picker permet de sélectionner une valeur parmi plusieurs possibilités :

```
struct CouleurView: View {
    @State private var couleur = "Rouge"

    var body: some View {
        Picker(
            "Couleur",
            selection: $couleur
        ) {
            Text("Rouge")
                .tag("Rouge")
            Text("Vert")
                .tag("Vert")
            Text("Bleu")
                .tag("Bleu")
        }
    }
}
```

- selection: $couleur est un Binding, comme vu avec l'état : la variable est mise à jour lorsque l'utilisateur choisit une option
- .tag associe une valeur à chaque option ; son type doit être celui de la variable (ici String)
- Dans un Form, SwiftUI adapte automatiquement la présentation

3. Picker avec une énumération

Dans une vraie application, on utilise une énumération. Avec CaseIterable, Swift génère Priorite.allCases, et une propriété libelle fournit le texte à afficher :

```
enum Priorite: String, CaseIterable {
    case basse
    case moyenne
    case haute
    case critique

    var libelle: String {
        switch self {
        case .basse: return "Basse"
        case .moyenne: return "Moyenne"
        case .haute: return "Haute"
        case .critique: return "Critique"
        }
    }
}
```

```
struct PrioriteView: View {
    @State private var priorite: Priorite = .moyenne

    var body: some View {
        Picker(
            "Priorité",
            selection: $priorite
        ) {
            ForEach(
                Priorite.allCases,
                id: \.self
            ) { valeur in
                Text(valeur.libelle)
                    .tag(valeur)
            }
        }
    }
}
```

- ForEach crée une option pour chaque valeur de allCases
- Le tag reçoit la valeur de l'énumération elle-même, et non son texte
- CategorieTicket, PrioriteTicket et StatutTicket, fournies dans le projet de départ, fonctionnent de la même façon : elles offrent allCases et libelle

4. Styles de Picker

Le modificateur pickerStyle change l'apparence du Picker :

Style	Utilisation
`.menu`	Menu déroulant, adapté à plusieurs options
`.segmented`	Barre de boutons, pour peu d'options courtes : Liste / Grille, Jour / Semaine, Actifs / Tous

```
Picker(
    "Affichage",
    selection: $mode
) {
    Text("Liste")
        .tag("Liste")
    Text("Grille")
        .tag("Grille")
}
.pickerStyle(.segmented)
```

5. Toggle

Toggle représente une valeur booléenne : activé, la variable vaut true ; désactivé, elle vaut false.

```
struct NotificationsView: View {
    @State private var notifications = true

    var body: some View {
        Toggle(
            "Notifications",
            isOn: $notifications
        )
    }
}
```

On peut ajouter une icône avec Label :

```
Toggle(
    isOn: $notifications
) {
    Label(
        "Notifications",
        systemImage: "bell"
    )
}
```

Un Toggle peut aussi piloter l'apparence, avec preferredColorScheme (mode clair ou sombre) :

```
Form {
    Toggle(
        "Mode nuit",
        isOn: $modeNuit
    )
}
.preferredColorScheme(
    modeNuit ? .dark : .light
)
```

6. Stepper

Stepper augmente ou diminue une valeur. Avec in:, la valeur reste dans une plage ; avec step:, on choisit l'incrément :

```
@State private var quantite = 1
@State private var note = 5.0

var body: some View {
    VStack {
        Stepper(
            "Quantité : \(quantite)",
            value: $quantite,
            in: 1...10
        )

        Stepper(
            "Note : \(note)",
            value: $note,
            in: 0...10,
            step: 0.5
        )
    }
}
```

Il est utile pour une quantité, un nombre de personnes ou une capacité.

7. DatePicker

DatePicker permet de sélectionner une date ou une heure. displayedComponents choisit ce qui est affiché :

Valeur	Affichage
`.date`	La date seulement
`.hourAndMinute`	L'heure seulement
`[.date, .hourAndMinute]`	La date et l'heure (valeur par défaut)

Avec in:, on limite la plage de dates. Ici, Date()... signifie : à partir d'aujourd'hui, donc aucune date passée :

```
DatePicker(
    "Date de livraison",
    selection: $date,
    in: Date()...,
    displayedComponents: .date
)
```

8. Réagir à la sélection

L'état contrôle directement la structure de l'interface. Un Toggle peut afficher ou masquer d'autres contrôles :

```
VStack {
    Toggle(
        "Ajouter une date limite",
        isOn: $avecDateLimite
    )

    if avecDateLimite {
        DatePicker(
            "Date limite",
            selection: $dateLimite,
            displayedComponents: .date
        )
    }
}
```

Un switch peut aussi afficher une information différente selon la valeur choisie :

```
switch priorite {
case .basse:
    Text("Traitement normal")
case .moyenne:
    Text("Traitement standard")
case .haute:
    Text("Traitement prioritaire")
case .critique:
    Text("Intervention immédiate")
}
```

9. Exemple — Formulaire de ticket

```
struct TicketFormView: View {
    @State private var priorite: Priorite = .moyenne
    @State private var notification = true
    @State private var dateSouhaitee = Date()

    var body: some View {
        Form {
            Section("Demande") {
                Picker(
                    "Priorité",
                    selection: $priorite
                ) {
                    ForEach(
                        Priorite.allCases,
                        id: \.self
                    ) { valeur in
                        Text(valeur.libelle)
                            .tag(valeur)
                    }
                }
            }

            Section("Options") {
                Toggle(
                    "Recevoir une notification",
                    isOn: $notification
                )
                DatePicker(
                    "Date souhaitée",
                    selection: $dateSouhaitee,
                    displayedComponents: .date
                )
            }
        }
    }
}
```

10. Réutiliser un Picker dans une sous-vue

Une vue enfant peut recevoir la sélection de la vue parent avec @Binding :

```
struct PrioritePickerView: View {
    @Binding var priorite: Priorite

    var body: some View {
        Picker(
            "Priorité",
            selection: $priorite
        ) {
            ForEach(
                Priorite.allCases,
                id: \.self
            ) { valeur in
                Text(valeur.libelle)
                    .tag(valeur)
            }
        }
    }
}

struct TicketView: View {
    @State private var priorite: Priorite = .moyenne

    var body: some View {
        PrioritePickerView(
            priorite: $priorite
        )
    }
}
```

Avec un ViewModel @Observable, la liaison s'écrit $viewModel.priorite (avec @Bindable), comme vu avec MVVM.

11. Picker et filtrage

Un Picker peut aussi servir à filtrer une collection. Une sélection optionnelle représente « aucun filtre » :

```
struct TicketsFiltreView: View {
    let tickets: [Ticket]
    @State private var filtre: Statut? = nil

    var ticketsFiltres: [Ticket] {
        guard let statut = filtre else {
            return tickets
        }
        return tickets.filter { $0.statut == statut }
    }

    var body: some View {
        Picker(
            "Statut",
            selection: $filtre
        ) {
            Text("Tous")
                .tag(Statut?.none)
            ForEach(
                Statut.allCases,
                id: \.self
            ) { statut in
                Text(statut.libelle)
                    .tag(Statut?.some(statut))
            }
        }
        .pickerStyle(.segmented)
    }
}
```

- nil signifie Tous : la liste n'est pas restreinte
- Le tag doit avoir le même type que la sélection : Statut?.none et Statut?.some(statut)

12. Éviter la saisie libre inutile

À éviter :

```
TextField(
    "Priorité",
    text: $priorite
)
```

L'utilisateur pourrait écrire urgent, super haute, important ou abc. On préfère un Picker avec des valeurs définies (une énumération CaseIterable plutôt qu'une chaîne), ce qui garantit :

- Des données cohérentes
- Moins d'erreurs
- Une meilleure expérience utilisateur
- Une validation plus simple

13. À retenir

Contrôle	À retenir
`Picker`	selection et .tag ; ForEach(allCases) avec une énumération CaseIterable
`Toggle`	isOn lié à un Bool
`Stepper`	value, in: pour la plage, step: pour l'incrément
`DatePicker`	selection, displayedComponents, in: pour la plage
`Liaison`	@State, @Binding, @Observable, @Bindable

14. Exercice — Création d'un ticket

1. Énumérations

Créer les énumérations CategorieTicket et PrioriteTicket, conformes à CaseIterable, chacune avec une propriété libelle.

2. Formulaire

Créer une vue NouveauTicketView contenant un formulaire avec :

- Titre
- Description
- Catégorie
- Priorité
- Recevoir une notification
- Date souhaitée

Utiliser TextField, TextEditor, Picker, Toggle, DatePicker, Form et Section. Les Picker doivent être générés à partir de allCases.

15. Exercice — Date souhaitée facultative

Ajouter un Toggle « Ajouter une date souhaitée ». Le DatePicker ne doit apparaître que lorsque le Toggle est activé.